

Writing Functions and Using Built-In Functions

A One-Hour Introductory Lesson for New Programmers

Based on Python Illustrated, Chapter 6

Lesson Overview

Audience	Complete beginners (have covered variables, conditionals, and loops)
Duration	60 minutes
Format	Live-coding lecture with two short hands-on checkpoints
Source material	Python Illustrated, Chapter 6, "Writing Functions and Using Built-In Functions"

Learning Objectives

By the end of this lesson, students will be able to:

1. Explain why functions reduce duplicate code and make programs easier to maintain.
2. Define a function with `def` and call it.
3. Tell the difference between a parameter and an argument.
4. Write functions that return a value, and explain what happens when a function doesn't.
5. Use default parameters and keyword arguments.
6. Recognize `*args` and `**kwargs` and explain what problem they solve.
7. Explain local scope versus global scope, and why the `global` keyword should be used sparingly.
8. Use a handful of built-in functions and standard library modules (`math`, `random`, `datetime`).
9. Write a docstring.

Materials Needed

- Projector / screen with a Python editor (VS Code recommended, per Chapter 1 of the book)
- A Python interpreter students can follow along on, or a shared REPL (e.g., replit.com)
- The demo file `chapter6_functions_demo.py` (provided alongside this lesson)

Timed Agenda

#	Segment	Minutes
1	Why Functions Matter	5
2	Defining and Calling Your First Function	8
3	Parameters vs. Arguments	8

#	Segment	Minutes
4	Checkpoint 1 — Quick Practice	3
5	Return Values	9
6	Default Parameters & Keyword Arguments	7
7	A Peek at *args and **kwargs	4
8	Scope: Local vs. Global	7
9	Built-In Functions & the Standard Library	6
10	Docstrings	2
11	Wrap-Up & Homework	1
	Total	60

1. Why Functions Matter (5 min)

Talking points

Ask the class: “Has anyone ever copy-pasted the same few lines of code into three or four different places in a program?” Let that land — it’s exactly the problem functions solve.

- Functions let us bundle a block of code under one name and reuse it anywhere, as many times as we like.
- Without them, the same logic gets copy-pasted repeatedly. If a bug is found, or the logic needs to change, it has to be fixed in every copy — easy to miss one.
- Functions bring organization: each function has one job and a descriptive name, so the flow of a program reads almost like plain English.
- We’ve actually been using functions all along: `print()`, `input()`, and `range()` are all built-in functions.

Framing to use in class: think of a function as a magic spell. You write the incantation once (`def`), give it a name, and from then on you can cast it (call it) as many times as you like.

2. Defining and Calling Your First Function (8 min)

Talking points

- A function is declared with the keyword `def`, followed by a name, parentheses `()`, and a colon `:`.
- The indented block underneath is the function body — the code that runs when the function is called.
- Defining a function does not run it. Nothing happens until the function is called by writing its name followed by parentheses.

Live-code this:

```
def greet():  
    print("Hello humans!")  
  
# Nothing happens yet — the function has only been defined.  
  
greet() # Now it runs.
```

Instructor note: Deliberately run the code with only the `def` block first, and ask students to predict the output (there won’t be any). Then add the call and run again. This visually reinforces “defining ≠ running.”

3. Parameters vs. Arguments (8 min)

Talking points

- A parameter is a placeholder listed inside the parentheses of a function definition.
- An argument is the actual value you plug into that placeholder when you call the function.
- A common beginner mix-up: parameters and arguments describe the same “slot,” just at two different moments (writing the function vs. using the function). It’s fine to use the words loosely once the class is comfortable, but be precise for now.

Live-code this:

```
def greet_by_name(name):    # name is a parameter  
    print("Hello", name)  
  
greet_by_name("Zia")        # "Zia" is an argument
```

```
greet_by_name("Wiesje")    # "Wiesje" is an argument
```

Point out: name takes on a new value each time the function is called — that's the whole point of a parameter.

Multiple parameters — extend the example:

```
def feed_pet(pet_name, food):  
    print("Feeding", pet_name, "some delicious", food)  
  
feed_pet("Zia", "tuna")  
feed_pet("Wiesje", "kibble")
```

Multiple parameters are simply separated by commas, in both the definition and the call.

4. Checkpoint 1 — Quick Practice (3 min)

Have students write, in the shared editor or on their own machines, a function `describe_animal(name, sound)` that prints "<name> says <sound>!", then call it twice with different animals. Cold-call one or two students to share their output.

5. Return Values (9 min)

Talking points

- So far our functions only displayed something with `print()`. Often we want a function to produce a piece of data we can use elsewhere — that's what return is for.
- When Python hits a return statement, it immediately exits the function and sends that value back to wherever the function was called.
- If a function has no return statement, it returns a special value: `None`.
- `print()` and `return` are not the same thing — `print()` only displays something in the console; `return` hands a value back to the calling code so it can be stored or reused. This is one of the most common early confusions, so linger on it.

Live-code this:

```
def area_of_rectangle(length, width):  
    area = length * width  
    return area  
  
result = area_of_rectangle(5, 3)  # result now holds 15  
print("Area is:", result)
```

Then show the “no return” case:

```
def return_nothing():  
    print("Hi")  
  
val = return_nothing()  
print(val)  # None
```

Instructor note: ask “what do you think will print for `val`?” before running it. Most beginners guess wrong the first time — a good teaching moment.

6. Default Parameters & Keyword Arguments (7 min)

Talking points

- A default parameter lets you skip an argument when calling a function — Python falls back to a preset value.
- Keyword arguments let you name which parameter an argument belongs to when you call the function, so order stops mattering. This is mostly useful for readability when a function has many parameters.

Live-code this:

```
def greet_friend(name, greeting="Hello"):
    print(greeting, name)

greet_friend("Zia")          # Hello Zia (default used)
greet_friend("Wiesje", "Hi") # Hi Wiesje (default overridden)

def describe_pet(pet_name, species, age):
    print(f"{pet_name} is a {species} who is {age} years old.")

describe_pet(species="dachshund", pet_name="Wiesje", age=7)
```

Note for students: normally arguments are matched by position (first argument → first parameter, etc.). Keyword arguments override that by explicitly labeling each value.

7. A Peek at *args and **kwargs (4 min)

Talking points (keep this brief and conceptual — it's a preview, not mastery)

- Sometimes you don't know in advance how many arguments a function will receive.
- *args collects any extra positional arguments into a tuple.
- **kwargs collects any extra keyword arguments into a dictionary.

Live-code this:

```
def show_off(*args, **kwargs):
    print("Positional args:", args)
    print("Keyword args:", kwargs)

show_off("Zia", "Wiesje", toy="feather wand", snack="tuna")
# Positional args: ('Zia', 'Wiesje')
# Keyword args: {'toy': 'feather wand', 'snack': 'tuna'}
```

Tell students they'll see *args/**kwargs a lot when reading other people's code (and in later chapters), even before they're writing it themselves regularly.

8. Scope: Local vs. Global (7 min)

Talking points

- Scope is the part of the program where a name (variable or function) can be referred to.
- Variables created inside a function — including its parameters — are local: they exist only while the function runs and disappear afterward.
- Variables created outside any function (unindented, at the top level of the file) are global: accessible everywhere, including inside functions.

- If a variable exists in both scopes, the local one “wins” inside the function, but the global one is untouched outside it.

Live-code, in this order — the order matters for the “aha”:

1) A local variable can't be used outside its function

```
def feed_cat():
    food = "tuna"
    print("Feeding cat with", food)

feed_cat()
print(food)  # NameError! food only exists inside feed_cat
```

2) A global variable CAN be read inside a function

```
food = "kibble"

def feed_cat():
    print("Feeding cat with", food)

feed_cat()
print(food)
# Feeding cat with kibble
# kibble
```

3) But assigning inside a function creates a NEW local variable — it does not overwrite the global one

```
food = "kibble"

def feed_cat():
    food = "tuna"
    print("Feeding cat with", food)

feed_cat()
print(food)
# Feeding cat with tuna
# kibble <- global "food" is untouched
```

Then introduce global, with a caution:

```
food = "kibble"

def feed_cat():
    global food
    food = "tuna"
    print("Feeding cat with", food)

feed_cat()
print(food)
# Feeding cat with tuna
# tuna <- global "food" WAS changed
```

Talking point: global works, but it’s generally discouraged — it makes code harder to follow because a function can silently change something far away in the file. The better habit is to pass values in as parameters and return updated values, e.g. `food = feed_cat(food)`.

9. Built-In Functions & the Standard Library (6 min)

Talking points

- Beyond functions we write ourselves, Python ships with many built-in functions ready to use immediately — `print()`, `len()`, `type()`, `int()`, `float()`, `str()`, and more.
- Beyond that, Python's standard library offers entire modules of additional functions that must be imported before use. Show one or two per module — don't try to be exhaustive.

math module:

```
import math

print(math.sqrt(25))    # 5.0
print(math.pow(2, 3))   # 8.0
print(math.factorial(5)) # 120
print(math.pi)         # 3.14159...
```

random module:

```
import random

print(random.randint(1, 6))    # random int 1-6, like a dice roll
print(random.choice(["a", "b", "c"])) # random element from a list
```

datetime module:

```
from datetime import datetime

now = datetime.now()
print(now.year, now.month, now.day)
```

Talking point: mention that students will also learn to write their own reusable modules (files of functions they can import), but that's a natural next step after today — the book covers it as well, right in this same chapter, and it's worth a self-read.

10. Docstrings (2 min)

Talking points

- A docstring is a string in triple quotes `"""..."""` placed as the very first line inside a function body.
- It documents what the function does, so that other people — and future-you — understand its purpose without reading every line.
- Good editors (like VS Code) show the docstring in a tooltip when you hover over a function call.

```
def area_of_circle(radius):
    """Returns the area of a circle with a given radius, using pi * r^2."""
    import math
    return math.pi * radius * radius
```

Talking point: a one- or two-sentence docstring is enough for most functions. The goal is clarity, not length.

11. Wrap-Up & Homework (1 min)

Recap in one breath: functions let us name and reuse code; parameters/arguments pass data in; return passes data back out; default and keyword arguments make functions flexible; scope determines where a variable is

visible; Python's built-ins and standard library save us from reinventing the wheel; docstrings explain our functions to others.

Homework (from Python Illustrated, Chapter 6 exercises):

- 6.1 Temperature Conversion — write a function that converts Celsius to Fahrenheit by default, with a way to switch direction. Include a docstring.
- 6.3 Packing Lunch Boxes — write a function using `**kwargs` to print the contents of a lunchbox.
- 6.4 Sorting Hat — write a function that randomly assigns one of four Hogwarts houses using the random module.

Encourage students to run and modify the accompanying `chapter6_functions_demo.py` file before attempting the homework — every concept from today appears there with runnable, commented examples.

Instructor Tips

- The single biggest stumbling block in this chapter is usually the difference between `print()` (displays something) and `return` (hands a value back to be reused). Spend extra time there if the room looks uncertain — it's worth slowing down for.
- Local vs. global scope is the second-biggest stumbling block. The three-step live-code sequence in Section 8 is designed to build the “aha” gradually — resist the urge to skip straight to the global keyword.
- If time is short, `*args/**kwargs` (Section 7) and the standard-library tour (Section 9) can be compressed to a single example each without losing the core idea — save extra time for Sections 5 and 8 instead.